



An Enhancement Proposal: Gemini CLI

OMMKPN NotFound 404 Error

Ming Yuan (Team Lead)	–	22xb63@queensu.ca
Michael Liu (Presenter)	–	22jdn2@queensu.ca
Omar Kaddah	–	22chm1@queensu.ca
Enrong Pan (Presenter)	–	enrong.pan@queensu.ca
Nandan Bhut	–	22gp44@queensu.ca
Kosi Amobi-Oleka	–	22fnbn@queensu.ca

Abstract

Gemini CLI is an open-source AI agent developed by Google that brings the capabilities of the Gemini large language model directly into the terminal. This report proposes a Full-Session Recording and Reusable Memory enhancement that enables the system to learn from past sessions by distilling user feedback and session activity into structured, locally stored memory entries that can be retrieved and injected into future prompts. We identify a key limitation in the current architecture: all session knowledge is either lost when a session ends or requires manual authoring through static GEMINI.md files. To address this, we present two architecturally distinct implementation approaches, one distributing trace distillation across all existing subsystems and the other centralizing it in a new Trace Manager component, and evaluate them through an SEI SAAM analysis against non-functional requirements including maintainability, testability, evolvability, and privacy. Based on this analysis, we recommend the centralized Trace Manager approach for its cleaner architectural boundaries, simpler testing surface, and lower coordination cost. On top of presenting two usecases that exemplify the enhancement, the report also examines the enhancement's impact on the concrete architecture at the subsystem, directory, and file levels, discusses potential risks, and outlines testing plans for verifying integration with existing features such as the ReAct loop, tool execution, and configuration loading.

1 Introduction

1.1 Derivation Process

Our starting point was the concrete architecture we recovered in A2. While reviewing the seven-component structure and its dependency relationships, we noticed that the Configuration System handles session history and GEMINI.md context files, but all of this is either lost when a session ends or requires the user to manually write project instructions. There is no way for the system to learn from past sessions automatically. We kept running into this limitation ourselves while using the CLI for testing during A2: every time we started a new session, we had to re-explain the same project context and preferences. This led us to ask whether the architecture could support some form of persistent, user-curated memory across sessions.

We confirmed with the Gemini CLI documentation[1] and codebase [2] that no such feature exists. The existing conversation history compression works only within a single session, and GEMINI.md files are static documents that the user writes by hand. Neither mechanism captures what actually happened during a session or lets the user decide what was worth remembering for next time.

Once we settled on cross-session memory as our proposed enhancement, we brainstormed how to realize it and came up with two architecturally different approaches. Approach A distributes the feature by adding a Trace Distiller subcomponent to each existing subsystem, so every component independently observes and summarizes its own activity. Approach B introduces a single new Trace Manager component that handles all tracing, compression, and memory storage in one place. We wrote up both approaches, then performed a SAAM analysis comparing them against the non-functional requirements most relevant to our stakeholders[3]. Based on that analysis, we determined that Approach B was the stronger fit, and only then did we produce the updated concrete architecture diagram reflecting the chosen implementation.

With the architecture finalized, we evaluated the enhancement's impact at multiple levels: how it changes the high-level concrete architecture and component dependencies, what happens at the subsystem level within affected components, and which specific directories and files in the codebase would need to be created or modified. We also examined how the new feature interacts with existing features such as the ReAct loop, tool execution, and GEMINI.md context loading to make sure it integrates without breaking current behavior.

1.2 Overview of New Feature and Some Motivations

Building upon the derivation, we can now provide a general overview of the new feature: Cross-session memory. As previously mentioned, this feature is designed to extend the system beyond its current session-bound behavior by allowing it to retain and reuse useful information across multiple sessions.

Currently, Gemini CLI operates within a single-session context window. While mechanisms such as session history and GEMINI.md files provide some support for maintaining context, they are not persistently stored nor automatically updated[1]. As a result, users must repeatedly restate project details, preferences, and workflows each time they begin a new session. This limitation prevents the system from leveraging prior interactions to improve future performance.

The newly proposed feature addresses this limitation by introducing a memory layer that captures and reuses valuable session information. At a high level, this new feature works by observing and recording key aspects of a user's session, such as task flows, tool usage patterns, and important decisions. Rather than storing raw logs, this information is processed into a structured and reusable form. It is important to note that the goal of this feature is not to record everything, but to preserve information that is most beneficial for enhancing future interactions. For instance, the system may reuse previously successful workflows, recall preferred ways of interacting with tools, or use past interactions as guidance when constructing new responses. By doing this, the CLI can reduce redundant steps, avoid repeating mistakes, and provide outputs that are better aligned with the user's wants and needs.

Another important aspect of this feature is that it should be user-aware and user-controlled. This means that users should be able to influence what information is retained, ensuring that only relevant data becomes part of long-term memory. This is achieved by prompting users during their sessions about whether they would like to retain the recently used information, workflows, or conditions for future sessions. For example, consider a user planning a marketing campaign across multiple sessions. During one session, the user creates a target audience list and a few draft email copies. The system may prompt: “Would you like to retain this data for future sessions?” If accepted, the system remembers the audience list and draft emails, allowing the user to continue working on the campaign in a later session without recreating prior work; thus saving time and maintaining consistency across sessions. This also helps maintain transparency and prevents the system from storing unnecessary or sensitive information without oversight.

From a system perspective, this feature introduces a new capability without fundamentally altering the existing interaction model. The CLI still operates in the same way during a session, but has now evolved to be able to carry over knowledge between sessions. This ensures that the enhancement integrates naturally with existing workflows without causing disruption.

2 Implementation Approaches

2.1 Enhancement Motivation and Benefits

We identified a key limitation in Gemini CLI: its effectiveness is bounded by a single context window. Once a session ends or a new one begins, previously accumulated context and experience are lost, preventing the system from leveraging prior work in future interactions. To address this, we propose reusable full-session memory (a mechanism that allows Gemini CLI to persist and reuse knowledge across sessions). This enables the system to take shortcuts such as skipping task decomposition, reusing prior outputs, or leveraging past interactions as examples for prompt construction (e.g., when interacting with APIs). As a result, the CLI can move more efficiently toward high-quality outputs without recomputing from scratch.

2.2 Implementation Approach A: Distributed Trace Distillers

In this approach, every existing subsystem (Terminal Interface, Orchestration Engine, Toolbox, Extension Manager, Task Delegation System, Security and Execution, and Configuration System) receives a Trace Distiller subcomponent. Each distiller independently observes and summarizes what its host subsystem did during a session using rule-based heuristics. At session end, these summaries are forwarded to the Orchestration Engine, which bundles them with user feedback collected through the Terminal Interface. The Orchestration Engine then sends the combined data to the Gemini API for a final distillation pass, and the resulting structured memory entries are stored locally by the Configuration System.

2.3 Implementation Approach B: Trace Manager

In this approach, we introduce a new component called the **Trace Manager** on top of the current Gemini CLI system. The Trace Manager is responsible for recording how the system operates during and across sessions. It consists of three main sub-components: the **Session Tracer**, which temporarily records the activity of major system components such as the Orchestration Engine, Toolbox, Extension Manager, Task Delegation System, and Configuration System throughout a session. These will be cleared if user decide not to keep it for long term or be handed over to Trace Compressor to compress them into structured memory; the **Trace Compressor**, which processes and compresses the collected trace into a more reusable form using user feedback organized by the Orchestration Engine with Gemini AI; and the **Persistent Memory Store**, which saves selected compressed traces or learned patterns as adaptive long-term memory for future sessions. Unlike the static instructions stored in the Configuration System, this memory evolves over time and is managed by rules defined in Configuration System such as when to free space and how to prioritize deletion of the less frequent use ones.

3 SEI SAAM Analysis

Our Software Architecture Analysis Method (SAAM) evaluates the architectural impact of the proposed Full-Session Recording and Reusable Memory feature by comparing two implementation approaches[3]. Approach A distributes trace distillation across all existing subsystems by embedding a Trace Distiller subcomponent into each one, while Approach B introduces a single new Trace Manager component that centralizes all tracing, compression, and memory responsibilities. By assessing how each approach affects key non-functional requirements from the perspectives of both users and developers, we determine which realization is the stronger overall fit for the Gemini CLI architecture.

3.1 Stakeholder Analysis

Users: The primary users of Gemini CLI are software developers and technical professionals who rely on the tool for AI-assisted coding, file manipulation, and task automation directly from the terminal. For these users, the Full-Session Recording and Reusable Memory feature represents a significant improvement in workflow continuity. Currently, every new session starts from scratch, and users must re-explain context, preferences, and project conventions each time. Users expect the memory system to work transparently in the background, surfacing relevant past experience without requiring manual setup or slowing down the session. Performance and responsiveness are critical[4]: any latency introduced by trace collection or memory retrieval during the ReAct loop would directly interrupt the fast, iterative workflow that makes the CLI valuable. Users also care deeply about privacy, since session traces may contain sensitive code, file paths, API keys, or proprietary project details. They expect all data to remain local on their machine, with clear control over what is remembered and what is deleted. In short, users want a system that learns from past sessions reliably and privately, without degrading the interactive experience they depend on.

Developers: For the developers maintaining Gemini CLI, the introduction of cross-session memory must be architecturally clean and sustainable. They are focused on maintainability and testability: adding memory capabilities should not scatter new logic across the entire codebase in ways that create hidden coupling or make future changes risky. Developers need the feature to be modular enough that trace collection, compression, storage, and retrieval can each be understood, tested, and modified independently. They also value evolvability[4], since the memory system will likely need to adapt as the Gemini model's capabilities grow, as new tools are added to the Toolbox, or as MCP extensions introduce new kinds of interactions worth remembering. A design that is tightly woven into every existing subsystem imposes a higher coordination cost for any future change, whereas a more centralized design localizes those changes. Additionally, developers consider the operational cost of the feature: approaches that require multiple concurrent Gemini API calls for distillation across components raise concerns about rate limiting, billing, and resource contention on the Node.js async loop. Developers seek an implementation that balances power with simplicity, keeping the architecture coherent as the system scales.

3.2 Non-Functional Requirements (NFRs) Analysis

We evaluate both approaches against seven non-functional requirements identified as most important to the stakeholders described above. For each NFR, we assess the impact on a three-point scale: **High** (the approach strongly satisfies the requirement), **Medium** (partial satisfaction with notable trade-offs), and **Low** (the approach poorly satisfies or actively works against the requirement)[3].

Approach A: Distributed Trace Distillers.

This approach has the advantage of first-hand domain knowledge: each distiller operates within the subsystem it observes, giving it direct access to the most relevant context. The Toolbox distiller can track exactly which tool parameters succeeded or failed, the Security distiller can record which policy decisions the user overrode, and the Extension Manager distiller can log which MCP tools were invoked and how they performed. This domain-specific awareness produces higher-fidelity raw traces compared to an external observer.

However, the distributed nature introduces significant costs. Six of the seven subsystems now contain new trace-related logic that must be maintained, tested, and evolved in coordination. If the memory schema changes or the distillation strategy is updated, every distiller across every subsystem must be modified consistently. The approach also raises concurrency concerns: multiple distillers may be summarizing data simultaneously at session end, and if several subsystems independently request Gemini API calls for AI-assisted distillation, this creates queuing and potential rate-limiting issues. Furthermore, testing requires verifying not just that each distiller works in isolation, but that the combination of all distillers produces coherent, non-redundant memory entries when their outputs are merged.

Approach B: Centralized Trace Manager.

This approach collects all session traces within a single Trace Manager rather than distributing tracing logic across subsystems. While it may experience slight delays or loss of fine-grained details, it ensures consistency and manageability by capturing data in a uniform format. Encapsulation simplifies testing, debugging, and future updates, and only a single API call at session end is needed to process user feedback and compress memory. Implementation to existing system is faster and integration is simpler due to the centralized design. However, centralization introduces risks: a bug or vulnerability could affect the entire feature, and handling most processing in one component may add workload overhead and impact performance.

The following table summarizes how each approach performs against the identified NFRs:

Table 1: NFR Comparison: Distributed Distillers vs. Centralized Trace Manager

NFR	A: Distributed Distillers	B: Centralized Trace Manager
Performance	Medium: Lightweight per-subsystem logging during a session, but end-of-session contention when multiple distillers forward summaries and potentially queue concurrent Gemini API calls simultaneously.	Medium This improvement comes with additional runtime and storage overhead for tracking. Though shortened the time needed at the end of the session to compress session memory.
Usability	Medium: Domain-specific memories are high quality (e.g., Security distiller captures approval preferences), but uncoordinated distillers risk producing overlapping or inconsistent entries that confuse the model.	High By centralizing session tracing, compression, and adaptive memory into a single component, the system can provide more consistent behavior, easier configuration, and smoother continuity across sessions.
Privacy	Medium: All data stays local, but privacy filtering (e.g., excluding API keys) must be implemented independently in each of the six distillers, risking inconsistent enforcement.	Medium: Privacy filtering can be enforced consistently across all types of information using the same standards. Though there is concentration risk, as a exploit in the Trace Manager could expose a large amount of sensitive data at once.
Evolvability	Low: Schema or strategy changes require simultaneous updates across all subsystems. Adding a new subsystem (as with Task Delegation after A1) requires creating and integrating a new distiller. Feature logic is coupled to the component count.	High: Updates can occur entirely within the Trace Manager component. Any new sub-component can be added without modifying core logic in other components, enabling easy evolution and extension.

NFR	A: Distributed Distillers	B: Centralized Trace Manager
Maintainability	Low: Trace logic scattered across six subsystems. A bug in the distillation schema or storage format requires coordinated fixes in multiple locations, increasing maintenance burden disproportionately.	High: Since the Trace Manager centralizes all logic for this feature, debugging is straightforward and localized. Maintenance is simplified because core feature logic is encapsulated within a single component.
Testability	Low: Individual distillers are unit-testable, but verifying coherent, non-redundant combined output requires end-to-end tests across all subsystems. Mocking six distillers is significantly more effort than testing one component.	High: With all logic centralized in the Trace Manager, testing requires only a few mocked calls to external components. There is no need to coordinate across multiple subsystems, simplifying unit and integration tests.
Reliability	High: First-hand domain context yields high-fidelity traces. If one distiller fails, others continue independently, providing graceful degradation with no single point of failure.	Low: Because the Trace Manager centralizes all core logic of the feature, a failure in the sub-component could impact the feature as a whole.
Concurrency	Low: Multiple distillers may summarize and request API calls concurrently at session end, causing resource contention on the Node.js async loop and potential rate-limiting issues.	High: Centralizing trace-related operations reduces conflicting API calls and duplicated processing. Concurrent trace collection introduces relatively little overhead, since it mainly involves recording information rather than performing heavy processing.

3.3 Comparison of Approaches

Both approaches achieve the same user-facing goal: enabling Gemini CLI to learn from past sessions through structured, locally stored memory entries. However, they differ substantially in how they distribute responsibilities across the architecture, and these structural differences have cascading effects on nearly every non-functional requirement.

Approach A’s primary strength is **reliability through domain proximity**. Because each distiller operates within its host subsystem, it has direct access to the richest possible context. The Toolbox distiller can observe exact tool parameters and error states; the Security distiller can capture nuanced policy override patterns; the Extension Manager distiller can record MCP server behavior that an external observer might not fully understand. This first-hand observation also provides natural fault isolation: if the Extension Manager’s distiller encounters an error, the Toolbox and Security distillers continue operating independently, ensuring partial trace collection even under failure conditions.

However, this distributed design imposes substantial costs in **maintainability, testability, and evolvability**. Trace-related logic is replicated across six subsystems, meaning that schema changes, distillation strategy updates, or privacy policy modifications require coordinated edits in multiple locations. This is the same pattern that made our A2 reflexion analysis difficult: when logic is spread across components, understanding and modifying it requires navigating the entire codebase rather than a single module. Testing is similarly affected, since verifying coherent memory output requires exercising all distillers together in integration tests rather than testing a single well-defined interface. The approach also introduces **concurrency risks**: at session end, multiple distillers may attempt Gemini API calls simultaneously, creating queuing on the Node.js async loop and potential rate-limiting issues that do not exist in the current architecture.

Approach B addresses many of these concerns by centralizing all memory-related logic within a single Trace Manager component. Due to this centralized design, the Trace Manager performs significantly better in terms of **evolvability**, **maintainability**, and **testability**. New updates to the feature can be added directly within the Trace Manager without requiring coordinated changes across multiple subsystems. Maintenance and debugging are also simplified, since developers only need to focus on one dedicated component rather than tracing logic scattered throughout the system. Similarly, testing can be performed in a more controlled and isolated manner because one component is responsible for both high-level and low-level feature behavior. Most importantly, these improvements can be achieved without altering or deeply understanding the internal logic of other existing components, which saves both development time and implementation effort. In addition, since session memory compression occurs only at the end of a session, the Trace Manager requires only a single API call through the Orchestration Engine to process user feedback and generate reusable memory. This reduces the **concurrency risks** introduced in Approach A, since Approach B only collects information concurrently from all components, making better use of time without placing significant pressure on the system. As all trace-related operations are handled in one place, the resulting data format remains consistent and easier to manage.

However, this approach also introduces several trade-offs affecting both **Reliability** and **Performance**. Its centralized structure creates a single point of failure risk, meaning that even a small error in collected information may disrupt the entire feature until the issue is resolved. Furthermore, because the Trace Manager collects information indirectly through transmission from other components at the same time, the resulting traces may be less domain-specific and may lose fine-grained contextual details as well as having some work overhead at the same time. This can reduce the quality or specificity of the reusable memory produced compared to a more distributed, subsystem-aware approach.

3.4 Conclusion: Centralized Trace Manager is More Favorable

Based on our analysis, Approach B **Centralized Trace Manager** is the stronger realization of the Full-Session Recording and Reusable Memory feature. While Approach A the distributed one offers higher reliability through distributed, first-hand trace collection and natural fault isolation, these advantages are outweighed by the significant costs it imposes on maintainability, testability, evolvability, and concurrency management.

The fundamental issue with Approach A is that it couples the memory feature’s complexity to the number of subsystems in the architecture. Every existing component gains new trace-related responsibilities, and any future architectural change such as adding a subsystem, modifying a distillation strategy, updating the memory schema, all requires coordinated modifications across the entire codebase. This violates the principle of separation of concerns that our layered architecture is designed to uphold. The concurrency implications further weaken Approach A: multiple simultaneous Gemini API calls at session end introduce resource contention and cost overhead that scale with the number of subsystems, whereas Approach B consolidates this into a single, manageable API interaction.

Approach B localizes all memory logic in one component with well-defined interfaces to the rest of the system. Changes to the memory feature: new compression strategies, updated privacy policies, additional storage backends, can be implemented and tested within the Trace Manager without modifying the Orchestration Engine, Toolbox, or any other existing subsystem. This makes the feature easier to evolve as Gemini CLI grows, easier to test through focused unit and integration tests, and easier to maintain by future developers who need to understand the memory system without navigating six different subsystems.

For users, both approaches deliver the same core benefit of cross-session learning. The difference is largely invisible at the user level, which makes the developer-facing advantages of Approach B which are lower maintenance cost, simpler testing, cleaner architectural boundaries, and more predictable performance characteristics the decisive factors[4]. We therefore recommend Approach B for implementation.

4 Enhancement Impacts

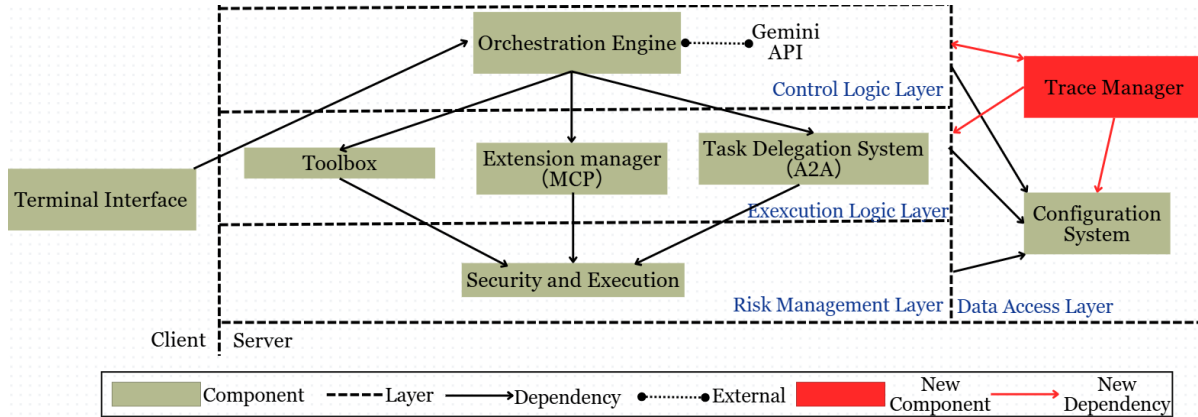


Figure: Conceptual Diagram with new component

4.1 Impacts on High Level Architecture

The transition from the conceptual architecture established in Assignment 2 to the multi-session memory enhancement involves expanding the system from seven subsystems to eight. The original model which comprised of the Orchestration Engine, Terminal Interface, Toolbox, Extension Manager, Task Delegation System, Security and Execution, and Configuration System[1], is joined by the Trace Manager. This component is centralized to record and manage system behaviour across multiple independent sessions. By localizing all cross-session memory responsibilities into a single module, the design maintains the hybrid client-server and layered architectural styles[5] that define the system's core structure.

Much like the configuration system, the trace manager is integrated as a cross-layer component that other components depend on. It primarily observes runtime activities within the Control and Execution Logic layers, ensuring the enhancement does not disrupt the system's core reasoning logic. This preserves the separation of concerns while enabling the CLI to evolve from a session-bound tool into a persistent, learning assistant.

There is a distinct bi-directional dependency between the Orchestration Engine and the Trace Manager: the engine relies on the Trace Manager to retrieve long-term context during initial prompt construction, while the Trace Manager depends on the engine to forward session activity data and execute the final Gemini API call for compression. The Trace Manager is designed to listen and act as a centralized listener where it receives updates from the Execution Logic Layer, such as current tools and parameters that have been asked for in the session from the toolbox or the inputs that were given by the user. It also listens and receives information in regard to the interactions and calls to external third-party extensions and tool schemas from the MCP. From the task delegation system, it receives notice on the success confirmation status in regards to whether or not it was successful in delegating tasks to the sub-agents. A one-way dependency exists where the Trace Manager relies on the Configuration System. The configuration system defines the rules for data storage and establishes the policies that govern deletion prioritization. Since the system is designed to store session data locally, the Trace Manager consults the Configuration System to determine how to manage and reduce storage consumption according to established limits.

4.2 Impacts On Subsystems

4.2.1 Terminal Interface

The introduction of the Trace Manager as an eighth component does not fundamentally alter its structural dependencies, as its primary interactions are through the Orchestration Engine, which acts as a gateway for forwarding requests. However, the behavior of the interface is significantly enhanced by the Session Tracer, where it receives data from the other components in the system. This data is eventually compressed

by the Trace Compressor and saved in the persistent memory store, allowing the terminal interface to evolve from a stateless entry point into a personalized environment that provides workflow continuity. At the end of a session, the interface captures user feedback for the Trace Compressor within the Trace Manager to prioritize the extraction of useful patterns while discarding irrelevant logs, a process that ensures sensitive information such as API keys or proprietary code is not stored without explicit user oversight.

4.2.2 Orchestration Engine

The Orchestration Engine remains the central server-side coordinator responsible for driving the ReAct loop and managing all communication, including the primary interface with the Gemini API. The addition of the Trace Manager assigns further responsibilities to the engine, allowing it to handle memory tasks across multiple sessions while keeping the core reasoning process intact. Since long-term memory is localized within the Trace Manager's Persistent Memory Store (PMS), the Orchestration Engine retrieves relevant patterns during initial prompt construction to speed up execution by skipping redundant task decompositions and looking up how it successfully resolved similar queries in the past. Throughout each session, the Session Tracer performs non-blocking, asynchronous observations of the engine's reasoning steps and tool calls, ensuring high-fidelity logs are recorded without impacting the responsiveness of the Node.js async loop. At the session's conclusion, the engine facilitates user-curated memory via the terminal interface, satisfying the requirement for RLHF oversight through feedback mechanisms like "thumbs up or down," and executes the final API call for the Trace Compressor to distill raw data into structured entries. By centralizing these cross-session operations, the system ensures that the Orchestration Engine's internal complexity remains manageable and does not scale with the volume of stored sessions, effectively evolving the CLI into a persistent, learning assistant.

4.2.3 Configuration System

The Trace Manager adds a dynamic and adaptive layer to the Configuration System's static configuration model. Entries sent to the Persistent Memory Store are governed by the Configuration System to ensure that adaptive memories complement, rather than hinder, the static instructions found in GEMINI.md files or pre-existing user settings. Furthermore, the persistent memory store utilizes the configuration system to manage its operational logic, determining which files and patterns are preserved in long-term memory for future sessions as the configuration system governs the policies regarding the prioritization of data deletion. Unlike the static information traditionally handled by the configuration system, the data within the persistent memory store evolves, necessitating rules to manage space and prioritize the deletion of less frequent patterns. While the Configuration System was previously limited to handling static user preferences and project instructions, it now governs the entire lifecycle of knowledge captured by the Trace Manager, ensuring that adaptive long-term memory adheres to the same hierarchical merge strategies as the rest of the CLI system.

4.2.4 Execution and Logic Layer

The Trace Manager has minimal impact on the operational logic of the Toolbox, Task Delegation System (TDS), and Extension Manager, as these components primarily function as broadcasters that forward data to the Session Tracer for long-term recording. The Toolbox is updated with a broadcast steps that sends a signal to Session Tracer that records the tool names, parameters, final output used when a tool is executed. The TDS also acts a broadcaster and sends the Session Tracer updates in regards to agent interactions by recording task handoffs and outcomes. The Execution Manager sends data in regards to any third-party tool or external extension that was required during the current session. This ensures that in future prompts the system has knowledge on what 3rd party tools it may need in the near future.

4.2.5 Risk Management Layer

The addition of the Trace Manager(TM) does not change how the Security and Execution layer functions. This layer acts as the system's gatekeeper, enforcing access controls and providing tool execution isolation via the Policy Engine. It also includes a sandbox that ensures defence-in-depth by managing sandboxes

via Docker, Podman, etc. On the contrary, the TM acts as a passive informational hub that does not execute tools or influence security decisions. The TM uses Session Tracer to get information in regard to the tools used by the ToolBox, it remains separate from the security logic layer.

4.3 Impacts On Directories and Files

The implementation of the Trace Manager primarily affects the server-side directory structures while also introducing modifications regarding the Orchestration Engine and Configuration System files.

File/Directory Added/Modified	Subsystem Affected
packages/core/src/trace-manager	Trace Manager
session-tracer.ts, persistent-memory.ts, trace-compressor.ts	Trace Manager
packages/core/src/agents/agent-scheduler.ts	Orchestration Engine
packages/core/src/agents/types.ts	Orchestration engine
packagaes.core/src/agents/agentLoader.ts	Orchestraton Engine
packages/core/src/tools/tool-registry.ts	Toolbox
packages/core/src/config/injectionService.ts	Configuration
packages/core/src/config/config.ts	Configuration

Table 2: Some Changes to Key Files

Files like session-tracer.ts, trace-compressor.ts, persistent-memory.ts were created to represent the three subcomponents in Trace Manager. The session tracer is primarily used for recording the run-time activities of other major subsystems, and the trace compressor was created to compress structured memory into a more usable form. The persistent memory. memory.ts is used to store long-term and dynamic memory. Agent-schedular.ts in the Orchestration Engine was modified to add custom code to trigger the end-of-session compression of files and forward the user feedback obtained from the feedback loop to the Trace Compressor. Types.ts was modified to add a shared interface for structured memories and trace patterns used by the Trace Compressor. The memory loop was updated to have the system record activities, handle user-approved retentions, and inject long-term context into new sessions. PromptConfig was modified to add a new field to hold the memories retrieved from the PMS to ensure long-term context is available during the ReAct loop[2]. AgentLoader.ts was updated to allow the PMS to inject cross-session context during the initialization of the agent via loadAgentsFromDirectory(). The agent schema was modified to include injectMemories to ensure structure integrity and to prevent the OE from running into errors during the construction of the initial prompt for a memory-aware agent. Tools-registry.ts we included a hook to the DiscoveredToolInvocation class where it captures the tool name and the intention of the call. This information is then sent to the Session Manager to store for future sessions. InjectionService.ts was updated to handle traceMemory from the PMS as now the model is able to distinguish the static code in GEMINI.md and the dynamic memory in PMS. To prevent repetitive memory from being stored, addInjection was modified to add a duplication check to ensure the model saves RAM and storage in the AI token window. config.ts[2] was expanded to incorporate the policies behind memory deletion, maximum memory allowed in Trace Manager. This was done through add a traceRetentionPolicy. We also added the an instance of a TraceManager component in Initalize() to ensure the Trace Manager is active before an agent is deployed.

5 Performance, Evolvability, Testability, and Maintainability

5.1 Performance

The new feature can improve performance[4] by reducing the time needed for similar future tasks, such as tool selection, task delegation, and workflow planning, since useful patterns are retained based on user feedback. This can lead to faster and more satisfactory outputs over time. However, because the feature must record the entire execution flow in order to generate reusable memory at the end of a session, it also

increases the amount of work the system must handle during execution a little bit.

5.2 Evolvability

The addition of this feature introduces some complexity to the system, since any future feature added to Gemini CLI would need to be considered in terms of whether, and how, its information should be recorded and stored for future use. As the system evolves[4], the Trace Manager may also need to be updated to adapt to new workflows or to support additional cross-session functionality. However, such updates are relatively easy to implement because they can be made within a single component, where all core logic is centralized in one place.

5.3 Testability

The new feature increases the number of feature interactions that the system must be tested against, since its behavior depends on information collected from multiple parts of the system. However, these interactions are relatively straightforward to test, as they mainly involve trace collection and data transfer between components[1].

5.4 Maintainability

The system becomes slightly more complex than the original design because it now includes a persistent memory store that must be maintained regularly. This maintenance is necessary to ensure that stored traces remain useful and relevant, so that the feature continues to improve system behavior rather than simply consuming storage without providing meaningful performance benefits. [4]

6 Potential Risks

Adding the Trace Manager to Gemini CLI introduces several risks that need to be considered before implementation. We elaborate on some potential risks below.

6.1 Single Point of Failure

Because the Trace Manager centralizes all memory-related logic into one component, any bug or crash in it takes down the entire feature. Unlike Approach A, where individual distillers fail independently and the rest continue collecting traces, a failure in the Trace Manager means no traces are recorded, no compression happens, and no past knowledge gets injected into future sessions. To mitigate this, the system should be designed for graceful degradation: if the Trace Manager encounters an error, the rest of the CLI should continue functioning normally as if the memory feature did not exist, rather than propagating the failure to the Orchestration Engine or Terminal Interface.

6.2 Storage Growth and Local Disk Pressure

Session traces and memory entries accumulate on the user's local machine over time. For users working on long-running projects with many sessions, this could consume a significant amount of disk space, especially if the Trace Compressor does not reduce data aggressively enough or if the retention policy is too generous. This risk is compounded by the fact that raw traces from the Session Tracer can be verbose before compression. To address this, the Persistent Memory Store should enforce configurable size caps, automatically prune the oldest or least-accessed entries when limits are reached, and provide clear user-facing commands for reviewing and managing stored data.

6.3 Sensitive Data Leakage into Memory

During a session, the model and tools may process sensitive information such as API keys, database passwords, file contents, or proprietary source code. If the Trace Compressor does not properly filter this content before writing to the Persistent Memory Store, sensitive data ends up stored on disk and could be injected into future prompts where it serves no purpose. A scrubbing pass should be applied before any trace data is persisted, stripping known sensitive patterns like credentials and tokens. Users should also be able to review individual memory entries and delete anything they are not comfortable retaining.

6.4 Memory Poisoning from Stale Knowledge

Distilled memories from past sessions may become outdated as a project evolves. For example, a memory might reference a file structure that has since been reorganized, or a tool preference that is no longer appropriate for the current codebase. If the Orchestration Engine injects stale memories into prompts, the model could make worse decisions than it would with no memory at all. To mitigate this, the Persistent Memory Store’s retention policy should track recency and access frequency, expiring entries that have not been used recently. However, this remains an inherent risk of any persistent memory system and requires careful tuning of the retention rules to balance between keeping useful knowledge and discarding outdated information.

7 Testing

To ensure that the Trace Manager integrates reliably with the existing Gemini CLI architecture, testing must cover both the correctness of the new component itself and its interactions with existing features. Given the architectural scope of the enhancement, testing will occur at two levels: unit testing of the Trace Manager’s subcomponents and integration testing across subsystem boundaries.

7.1 Unit Testing

Unit tests will verify the behavior of each subcomponent within the Trace Manager independently. The Session Tracer should be tested to confirm it correctly records events forwarded by the Orchestration Engine, including tool calls, model responses, and user feedback, and that it handles edge cases like very short sessions with no meaningful activity. The Trace Compressor should be tested for its ability to produce well-structured memory entries from raw traces, and critically, for its ability to strip sensitive data such as API keys and passwords before anything is written to storage. The Persistent Memory Store should be tested for basic read, write, and delete operations, as well as for enforcing storage caps, correctly pruning entries when limits are exceeded, and accurately tracking access frequency to support the retention policy.

7.2 Integration Testing with Existing Features

Integration testing will validate that the Trace Manager interacts correctly with the broader Gemini CLI system without disrupting existing behavior. The most important interaction to test is with the ReAct loop: memory retrieval at session start should not add noticeable latency to the first prompt, and injected memories should not interfere with the model’s normal tool selection and reasoning during the loop. Tool execution through the Toolbox should also be tested to confirm that the Session Tracer captures tool calls without affecting the execution flow or the confirmation dialogs managed by Security and Execution.

We also need to verify that injected memories coexist properly with GEMINI.md context files and user settings loaded by the Configuration System. Memories should complement, not conflict with or duplicate, the static instructions already present in these files, and the existing settings hierarchy should always take precedence. Additionally, the end-of-session compression flow requires testing for edge cases such as the user force-quitting mid-session before compression completes, and very long sessions where the raw trace exceeds the token window and must be chunked. Regression tests should confirm that existing CLI behavior is completely unchanged when the Trace Manager is present but no memories have been created yet, and when the memory feature is explicitly disabled through configuration.

8 Use Cases

8.1 Use Case 1

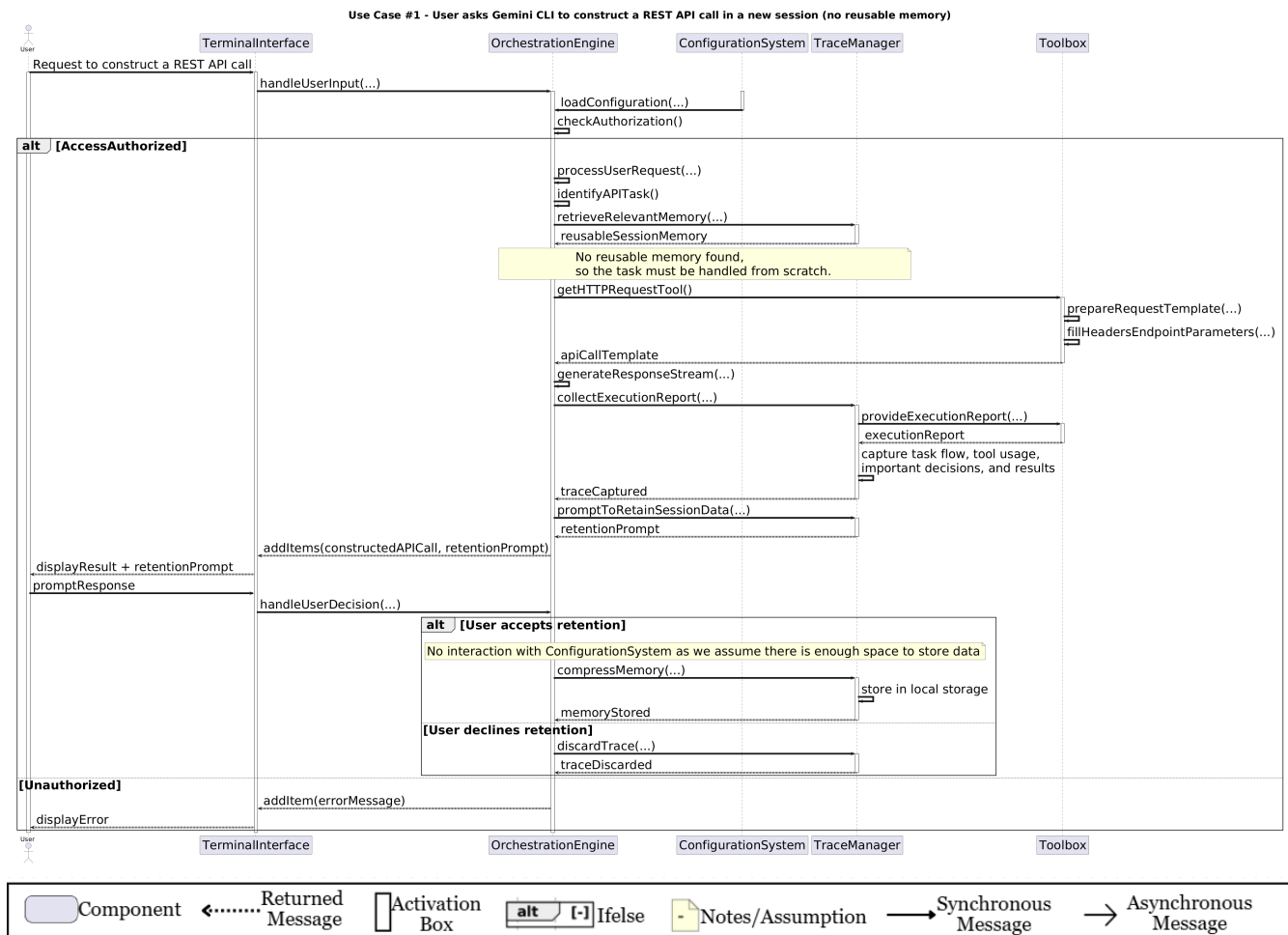


Figure: User asks Gemini CLI to construct a REST API call in a new session (no reusable memory)

For Use Case 1, the situation involves the first time Gemini CLI handles a request of a particular type. In this case, the TraceManager does not yet have the relevant information to streamline the task, so its role is instead to record the process for future reuse when a similar request is made. For the sake of clarity, the ExtensionManager, TaskDelegationSystem, and SecurityAndExecution components were removed from the diagram, as they are not used in this scenario. The diagram begins when the user submits a request to construct a REST API call through the TerminalInterface. This request is then passed to the OrchestrationEngine through `handleUserInput(...)`, where the system begins processing the user's input. The OrchestrationEngine then interacts with the ConfigurationSystem through `loadConfiguration(...)` to retrieve any necessary configuration or contextual information from the current session. Before any further processing occurs, the OrchestrationEngine checks authorization to Gemini services by calling `getAuthorizationStatus()` on itself. If authorization fails, the method call returns `unauthorized`, and the OrchestrationEngine sends an error message back to the TerminalInterface. If authorization succeeds, the OrchestrationEngine proceeds with `processUserRequest(...)` and `identifyAPITask()`, which interpret the request and determine the necessary operations. The OrchestrationEngine then attempts to retrieve any relevant reusable memory by calling `retrieveRelevantMemory(...)` on the TraceManager; however, since this is a new type of request, no reusable session memory is found, meaning the task must be handled from scratch rather than leveraging cross-session context. The OrchestrationEngine then calls `getHTTPRequestTool()` on the Toolbox to get the appropriate capability for constructing the API re-

quest. Within the Toolbox, `prepareRequestTemplate(...)` is called to create the structure for the API call, followed by `fillHeadersEndpointParameters(...)`, which fills the request with the necessary headers, endpoint details, and parameters. The completed API call template is then returned to the `OrchestrationEngine` as `apiCallTemplate`. Once the API call has been constructed, the `OrchestrationEngine` calls `generateResponseStream(...)` to prepare the final response. It then calls `collectExecutionReport(...)` on the `TraceManager` to record the execution process. The `TraceManager` then interacts with the `Toolbox` by calling `provideExecutionReport(...)`, which returns a comprehensive `executionReport` containing all details of the process. The `TraceManager` then analyzes this information to extract only the relevant elements, including the task flow, tool usage, important decisions, and the resulting output. This is the key interaction in the diagram, as our new component now records these important elements of the process. After the execution report is captured, the `OrchestrationEngine` is also tasked with asking the user whether they would like to store the current session information for future use. This is done by calling `promptToRetainSessionData(...)`, which tells the `TraceManager` to generate a prompt summarizing the information that would be stored. Both the constructed API call and the prompt are then sent back to the `TerminalInterface` through `addItem(constructedAPICall, retentionPrompt)`, where they are displayed to the user simultaneously. The system then waits for the user's decision, which is returned through `handleUserDecision(...)`. If the user accepts, the `OrchestrationEngine` calls `compressMemory(...)` on the `TraceManager` and stores the session data in local storage for potential reuse in future interactions. If the user declines, the `OrchestrationEngine` instead calls `discardTrace(...)`, ensuring that no session data is saved.

8.2 Use Case 2

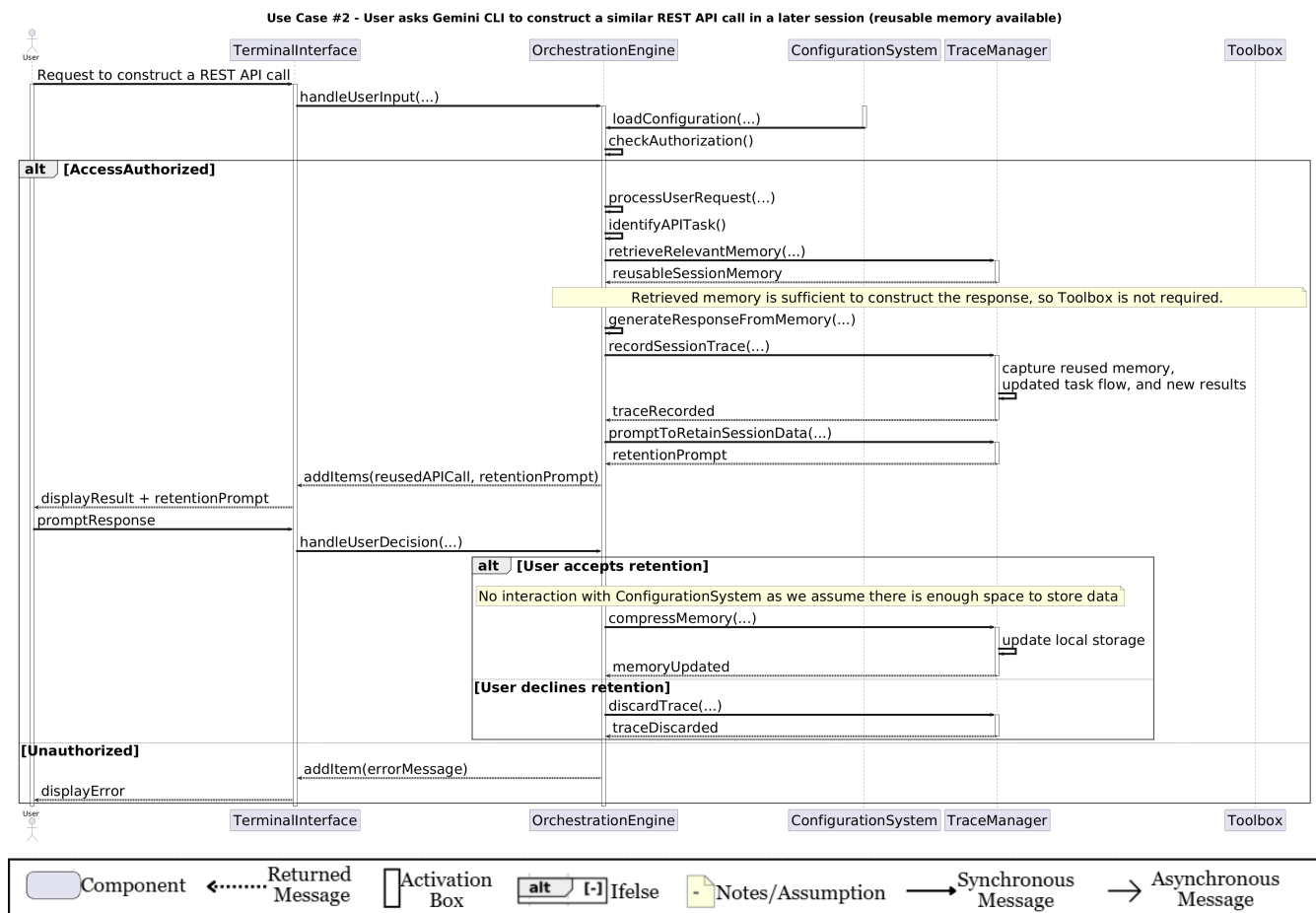


Figure: User asks Gemini CLI to construct a familiar REST API call (reusable memory available)

For Use Case 2, the situation involves a session in which Gemini CLI handles a request similar to one it

has previously processed, meaning reusable cross-session memory is now available. Up until `identifyAPITask()`, this sequence diagram flows exactly like Use Case 1. The new sequence events occur after the `OrchestrationEngine` calls `retrieveRelevantMemory(...)` on the `TraceManager` to access reusable session memory. In this case, since the request is familiar, relevant memory is successfully retrieved and the `reusableSessionMemory` is sufficient to construct the response, meaning the `Toolbox` is not needed. As a result, the `OrchestrationEngine` generates the response directly within itself by calling `generateResponseFromMemory(...)`, producing the API call. The `OrchestrationEngine` then records the updated session activity by calling `recordSessionTrace(...)` on the `TraceManager`, capturing the reused memory, updated task flow, and resulting output. After the trace is recorded, the `OrchestrationEngine` calls `promptToRetainSessionData(...)` to ask the user whether they would like to store the updated session information for future use. The `TraceManager` then generates a prompt summarizing the updated information and returns it to the `OrchestrationEngine`. Both the API call and the prompt are then sent back to the `TerminalInterface` through `addItem(reusedAPICall, retentionPrompt)`, where they are displayed to the user simultaneously. The system then waits for the user's decision, which is returned through `handleUserDecision(...)`. If the user accepts, the `OrchestrationEngine` calls `compressMemory(...)` on the `TraceManager` and updates the stored session data in local storage for future reuse. If the user declines, the `OrchestrationEngine` instead calls `discardTrace(...)`, ensuring that no new session data is saved. When comparing the sequence flow of a system with the `TraceManager` to one potentially without it, it is clear that the `TraceManager` significantly improves efficiency by allowing the system to reuse previously produced results through cross-session memory, rather than repeating the full execution flow. In a system without this capability, each request would need to be processed from scratch, requiring repeated interactions with components such as the `Toolbox`, `ExtensionManager`, or `TaskDelegationSystem`. However, the `TraceManager` avoids these unnecessary interactions, reducing both processing time and overall system complexity.

9 Lessons Learned

Proposing an architectural enhancement for a system as large as Gemini CLI was a different kind of challenge from recovering its conceptual or concrete architecture. The earlier reports focused on understanding what already exists; this one required us to reason about what should change and how. The lessons below reflect what we took away from this process.

The most important lesson was that there is no perfect implementation[3] for any feature. Every architectural choice involves trade-offs, and an approach that excels in one quality attribute will almost certainly sacrifice another. Our SAAM analysis made this very concrete: Approach A scored well on reliability because its distributed distillers provide fault isolation, but it performed poorly on maintainability, testability, and evolvability [4] for the same structural reasons. Approach B reversed this pattern, centralizing logic for easier maintenance and testing while introducing a single point of failure. Recognizing these trade-offs early and evaluating them systematically was far more productive than trying to find an approach that wins on every dimension.

We also learned that a proposed enhancement must be grounded in what stakeholders actually need, rather than what sounds technically interesting. It would be tempting to propose flashier features like real-time collaborative multi-agent sessions or automatic code refactoring driven by past session history, but these would introduce enormous complexity without clear demand from the users and developers who interact with the CLI daily. The Full-Session Recording and Reusable Memory feature, by contrast, addresses a specific and recurring pain point: the lack of cross-session continuity. Starting every session from scratch wastes time and forces users to repeat themselves, so the enhancement is motivated by a real usability gap rather than novelty for its own sake.

Finally, this assignment shows us that any enhancement should respect and work within the existing architecture rather than collapsing it. Throughout this assignment, we evaluated both approaches against the seven-component concrete architecture from A2 and made sure that the chosen implementation in-

tegrates through well-defined interfaces rather than rewriting or bypassing existing components. This reinforced why the conceptual and concrete architecture reports from A1 and A2 matter: without a clear understanding of the current system structure, it would be easy to propose changes that look reasonable in isolation but create tangled dependencies or violate the layered boundaries[5] the system relies on.

10 Conclusion

In conclusion, this report proposed a Full-Session Recording and Reusable Memory enhancement for Gemini CLI, addressing the limitation that all session knowledge is currently lost when a session ends or must be manually authored through static GEMINI.md files. We presented two architecturally distinct approaches: a distributed design embedding Trace Distillers into every existing subsystem, and a centralized design introducing a single Trace Manager component with three subcomponents. Through a SAAM analysis evaluating both methods on NFRs from the perspectives of users and developers, we determined that the centralized Trace Manager is the stronger realization, offering cleaner architectural boundaries, simpler testing and maintenance, and fewer concurrency risks at the cost of reduced fault isolation.

The chosen implementation adds the Trace Manager as a cross-layer component preserving the existing client-server and layered style, behaving as an optional observer with an internal pipe-and-filter structure across its three subcomponents. We examined the enhancement’s impact at the high-level architecture, subsystem, and directory levels, identified four potential risks with mitigations, and outlined testing plans to verify compatibility with existing features. Two use cases demonstrated how the system handles both new and familiar requests, illustrating the benefits of cross-session memory in reducing redundant work over time.

11 Dictionary and Naming Conventions

Term	Definition
GEMINI.md	Per-project markdown context files loaded by the Configuration System to provide project-specific instructions and context to the AI model.
ReAct Loop	(Reasoning and Acting Loop) The iterative cycle where the AI model reasons about a task, requests a tool action, observes the result, and repeats until the task is complete.
Stakeholders	Individuals or groups with an interest in the system’s design and behavior. In our analysis, this includes users (developers using the CLI) and developers (engineers maintaining the codebase).
SAAM	(Software Architecture Analysis Method) A structured evaluation technique that compares alternative architectural approaches by assessing their impact on stakeholder-identified non-functional requirements.
Trace Manager	The new component introduced in Approach B that centralizes session recording, trace compression, and persistent memory storage into a single architectural unit.
Trace Compressor	A subcomponent of the Trace Manager that processes raw session traces into concise, structured memory entries using user feedback and Gemini API-assisted summarization.
Persistent Memory Store	A subcomponent of the Trace Manager that saves compressed memory entries locally on the user’s machine and manages retention policies such as size caps and recency-based pruning.
Session Tracer	A subcomponent of the Trace Manager that records activity from major system components during a session, including tool calls, model responses, and user decisions.
Unit Testing	Testing individual subcomponents in isolation to verify their correctness independently of the rest of the system.
Integration Testing	Testing interactions between the new enhancement and existing system components to verify they work together without disrupting current behavior.

Table 3: Dictionary and Naming Conventions

12 AI Collaboration Report:

12.1 AI Member Profile and Selection Process:

For this assignment, our team decided to continue with using Gemini 3 Pro (Nov 2025 version) as our AI collaborator. Gemini performed well in both Assignment 1 and 2, and since the Gemini CLI is built on the same infrastructure as the Gemini model itself, it remained the most logical choice.

12.2 Tasks Assigned to the AI Teammate:

Our AI partner was used mainly for brain storming ideas, refinement tool and supporting our analysis when coming up with new ideas. It contributed to things such as:

- **Enhancement Ideas and Feasibility:** The AI was used to brainstorm potential feature enhancements for Gemini CLI and help determine which ideas were realistic to implement within the constraints of the already present system. It assisted with narrowing down which features to consider by evaluating complexity against the current component structure.
- **Alternative Design Exploration:** We used the AI to support the exploration of the implementation methods for the feature we selected. It helped us compare trade-offs between the distributed and centralized designs during the selection process.
- **Paragraph(s) refinement and Clarity:** Similar to the previous assignments, we used the AI to help identify grammatical errors, refine written sections by improving clarity, and articulate ideas more clearly.

12.3 Validation and Quality Control Procedures:

As in previous assignments, we treated all AI outputs as suggestions rather than answers and validated them through different methods. In this assignment, the AI was also used in more complex ways such as improving brainstorming results, identifying stakeholders, and exploring trade-offs in different implementations. These results were reviewed through discussions and ensuring consistency with architectural principles established in previous reports. When the AI assisted in generating scenarios for evaluation, these were manually looked over by our group to see if they were realistic and relevant to the system's operation. Overall, this validation process ensured that the AI contributions improved the report without compromising accuracy.

12.4 Interaction Protocol and Prompting Strategy:

Our interaction strategy from Assignment 1 has pretty much stayed the same for this report where we initially attempted to have each team member engage directly with the AI by creating their own prompts for their specific tasks. Then, we implemented a peer-review step, requiring the group to reach a consensus on whether a prompt was complete before execution.

For this report, we chose to implement the previous three-step plan plus a more structured and role oriented strategy while working with Gemini pro. Instead of just general broad prompts, we presented the AI as a software architecture reviewer or a system designer to guide it towards producing more technical and realistic responses. Similar to the previous report our prompting process follows an iterative refinement approach; where we begin with a high level query (such as proposing an enhancement or brain storming ideas), then add context such as descriptions, constraints, and specific criteria. We also incorporated targeted prompting, asking the AI to generate alternative ideas with pros and cons, which helped us explore a broader solution space before narrowing down our final approach. Finally, we implemented a feedback loop where outputs were reviewed and refined through follow up prompts to improve accuracy.

Example prompt:

Initial Query: As a specialized software architecture professor, you will rigorously analyze and explain the integration of a new Trace Manager component within the existing Gemini CLI architecture.

Context: The Trace Manager is presented as a centralized component that is in charge of tracing sessions, compressing traces and storing reusable memory across sessions. It is to interact mostly with the Orchestration Engine while maintaining compatibility with the chosen architecture style.

Task: Explain how the Trace Manager interacts with existing subsystems and how it preserves separation of concerns within the system.

Example Query: Explain the differences between a centralized Trace Manager approach and a distributed memory collection approach in the context of system maintainability and performance.

Example Response: The centralized Trace Manager ensures that all the logic related to memory management is contained within one single entity; which improves maintainability and testability. Since there is no need to make coordinated changes to multiple systems, the centralized approach ensures that there is minimal concurrency overhead, given that there can only be one API call to end the session. The distributed approach, however, means that each system can independently handle memory, which improves fault isolation but increases the coordination costs and performance problems.

12.5 Reflection on Human-AI Team Dynamics:

Integrating AI into the assignment was a continuation of the iterative process we developed in the earlier reports, but with a stronger focus on feature design and evaluation. Since we were already familiar with how Gemini works, there was no need to spend time experimenting with strategies for prompts, which enabled us to use the AI in a more specific way; such as defining the scope of the feature and constraints.

In the previous report, we primarily used the model to brainstorm and generate ideas for the content we needed to write. For this report we used it more so for design and analysis. This included considering alternatives in implementation, and understanding stakeholders and trade-offs in quality attributes like performance, maintainability, and reliability. The AI was helpful in pointing out risks and edge cases.

Similarly to the first report, the AI's involvement introduced additional work, in addition to making sure that the decisions made by the AI in terms of the design were consistent with the actual system and assignment constraints. However, our established guidelines from earlier assignments made this more efficient, with time saved on exploration and analysis outweighing the validation cost. Overall, AI contributed approximately 5–7% of the final deliverable content.

This report reinforced that effective collaboration with AI is valuable when it supports complex reasoning tasks such as evaluating and designing new features, identifying trade-offs, while still relying on human judgment for final decisions.

References

- [1] Google Gemini Team. “Gemini cli architecture overview.” Official Gemini CLI Documentation, Accessed: Jan. 28, 2025. [Online]. Available: <https://geminicli.com/docs/architecture/>.
- [2] Google, *Gemini cli*, version 1.0, Nov. 18, 2025. [Online]. Available: <https://github.com/google-gemini/gemini-cli>.
- [3] R. Kazman, L. Bass, M. Webb, and G. Abowd, “SAAM: A method for analyzing the properties of software architectures,” in *Proceedings of the 16th International Conference on Software Engineering (ICSE)*, Sorrento, Italy: IEEE Computer Society, 1994, pp. 81–90. DOI: 10.1109/ICSE.1994.296768. [Online]. Available: <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=798ec13c76bb233d2441ac00de675ca584de8418>.
- [4] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice* (SEI Series in Software Engineering), 4th ed. Addison-Wesley Professional, 2022, ISBN: 9780136886099.
- [5] D. Garlan and M. Shaw, “An introduction to software architecture,” Carnegie Mellon University, School of Computer Science, Tech. Rep. CMU-CS-94-166, 1994. [Online]. Available: https://userweb.cs.txstate.edu/~rp31/papers/intro_softarch.pdf.